



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ

DEPARTMENT OF TELECOMMUNICATIONS

SOFTWAREVÝ SYNTEZÁTOR

SOFTWARE SYNTHESIZER

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Klára Venhodová

VEDOUCÍ PRÁCE

SUPERVISOR

MgA. Michal Indrák, Ph.D.

BRNO 2025



Bakalářská práce

bakalářský studijní program **Audio inženýrství**
specializace Zvuková produkce a nahrávání
Ústav telekomunikací

Studentka: Klára Venhodová

ID: 240226

Ročník: 3

Akademický rok: 2024/25

NÁZEV TÉMATU:

Softwarový syntezátor

POKYNY PRO VYPRACOVÁNÍ:

Navrhnete digitální zvukový syntezátor kombinující aditivní a subtraktivní syntézu s dalším typem syntézy dle vlastního výběru. Syntezátor nejprve implementujete a otestujete v prostředí Matlab, nejprve jako funkce generující zvukové soubory a následně, pomocí Audio toolboxu, jako modul pracující v reálném čase. Otestovaný a odladěný algoritmus implementujete jako VST plug-in modul pomocí frameworku JUCE. Následně vytvořte šablonu pro implementaci jednoduchých syntezátorů v JUCE a vytvořte návod k jejímu použití.

DOPORUČENÁ LITERATURA:

- [1] RUSS, Martin. Sound synthesis and sampling. 2nd ed. Amsterdam: Elsevier, 2004. ISBN 0-240-51692-3.
- [2] REISS, Joshua D. a MCPHERSON, Andrew P. Audio effects: theory, implementation and application. Boca Raton: CRC Press, 2014. ISBN 978-1-4665-6028-4.

Termín zadání: 10.2.2025

Termín odevzdání: 3.6.2025

Vedoucí práce: MgA. Michal Indrák, Ph.D.

doc. Ing. Jiří Schimmel, Ph.D.
předseda rady studijního programu

UPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č.40/2009 Sb.

ABSTRAKT

Cílem práce bylo vytvořit digitální syntezátor a implementovat jej jako VST plug-in. Syntezátor je schopen 5 typů syntéz – aditivní, subtraktivní, amplitudové, kruhové a frekvenční modulace. První část práce byla provedena v Matlabu. Syntezátor nejprve pracoval jako generátor zvukových souborů podle uživatelem zadaných parametrů a následně byl upraven pro práci v reálném čase pomocí Matlab modulu Audio toolbox. Druhá část práce zahrnovala implementaci syntezátoru jako VST plug-in nástroj v jazyce C++ s využitím frameworku JUCE. V rámci práce byla zhotovena šablona pro implementaci jednoduchých syntezátorů pomocí JUCE frameworku a návod k jejímu použití.

KLÍČOVÁ SLOVA

frekvenční spektrum, spektrální složky, MATLAB, modulace, postranní pásmo, syntéza, filtr, VST, JUCE, C++, vzorek, blok

ABSTRACT

The aim of this thesis was to create a digital synthesizer and implement it as a VST plug-in. The synthesizer is capable of 5 types of synthesis – additive, subtractive, amplitude, ring and frequency modulation. The first part was implemented in Matlab. At first, the synthesizer generated sound files defined by parameters assigned by the user and was later adjusted to work in real time with the use of the Matlab module Audio toolbox. The second part focused on implementation of this synthesizer as a VST plug-in instrument, using the C++ language and JUCE framework. A template for implementing basic synthesizers in JUCE framework with instructions was created as part of this thesis.

KEYWORDS

frequency spectrum, spectral components, MATLAB, modulation, sideband, synthesis, filter, VST, JUCE, C++, sample, block

VENHODOVÁ, Klára. *Softwarový syntežátor*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2025. Vedoucí práce: MgA. Michal Indrák, Ph.D.

PODĚKOVÁNÍ

Ráda bych poděkovala vedoucímu bakalářské práce panu MgA. Michalu Indrákovi, Ph.D. za odborné vedení, konzultace, trpělivost a podnětné návrhy k práci.

Obsah

Úvod	11
1 Teoretická část studentské práce	12
1.1 Generování periodických průběhů	12
1.1.1 Čítač modulo	12
1.1.2 Integrátor a diferenciátor	13
1.1.3 Rekurzivní oscilátory	13
1.1.4 Přímá digitální syntéza	14
1.2 Aditivní syntéza	14
1.3 Subtraktivní syntéza	15
1.3.1 Filtrace při subtraktivní syntéze	15
1.4 Frekvenční modulace	15
1.4.1 Modulační index a deviační poměr	15
1.4.2 Frekvenční pásma	16
1.4.3 Lineární FM	17
1.4.4 Exponenciální FM	17
1.4.5 Fázová modulace	17
1.5 Amplitudová modulace	17
1.5.1 Modulační index	18
1.5.2 Frekvenční pásma	18
1.6 Kruhová modulace	19
1.7 Kmitočtové filtry	19
1.7.1 Parametrické filtry 1. řádu	19
1.7.2 Parametrické filtry 2. řádu	20
1.7.3 Hřebenový filtr	21
1.8 LFO	21
1.9 ADSR obálka	21
1.10 Generátor šumu	23
1.10.1 Bílý šum	23
1.10.2 Růžový šum	23
1.10.3 Červený šum	24
1.10.4 Azurový šum	24
1.10.5 Purpurový šum	25
2 Implementace syntezátoru v prostředí MATLAB	26
2.1 Generování zvukových souborů	26
2.1.1 Jednoduchý oscilátor	26

2.1.2	Filtry	26
2.1.3	Implementace syntéz	27
2.1.4	Generátor šumu	27
2.1.5	LFO	28
2.2	Implementace pracující v reálném čase	28
2.2.1	Jednoduchý oscilátor	28
2.2.2	Implementace filtrů	29
2.2.3	Implementace syntéz	29
2.2.4	Generátor šumu	29
2.2.5	Uživatelské rozhraní	30
3	Implementace syntezátoru v jazyce C++	31
3.1	Jazyk C++	31
3.2	JUCE framework	31
3.2.1	Projucer	31
3.3	Struktura	32
3.3.1	Generování signálu	32
3.3.2	Modifikátory signálu	33
3.3.3	Syntézy	34
3.3.4	Zpracovávání signálu v reálném čase	35
3.4	Uživatelské rozhraní VST plug-inu	36
3.4.1	Vykreslování komponentů uživatelského rozhraní	37
3.4.2	Napojení komponentů uživatelského rozhraní	38
3.5	Šablona pro implementaci jednoduchých syntezátorů v JUCE	38
	Závěr	40
	Literatura	41
	Seznam symbolů a zkratk	43
	Seznam příloh	44
	A Kód MATLAB implementace syntezátoru	45
	B Zdrojový kód a VST3 plug-in syntezátoru	46
	C Zdrojový kód šablony s pdf souborem návodu	47

Seznam obrázků

1.1	Besselovy funkce 1. druhu n-tého řádu	16
1.2	ADSR	22
1.3	Výkonové spektrum bílého šumu	23
1.4	Výkonové spektrum růžového šumu	24
1.5	Výkonové spektrum červeného šumu	24
1.6	Výkonové spektrum azurového šumu	25
1.7	Výkonové spektrum purpurového šumu	25
2.1	Blokové schéma návrhu Matlab syntezátoru	28
2.2	MATLAB uživatelské rozhraní syntezátoru	30
3.1	Blokové schéma návrhu VST syntezátoru	31
3.2	Uživatelské rozhraní VST syntezátoru	37

Seznam tabulek

1.1	Generování signálů integrátorem a diferenciátorem	13
1.2	Koeficienty filtrů 1. řádu.	20
1.3	Koeficienty filtrů 2. řádu.	20

Seznam výpisů

Úvod

Cílem práce je navrhnout a zrealizovat digitální syntezátor kombinující aditivní a subtraktivní syntézu s dalším typem syntézy dle vlastního výběru. Implementovat jej v MATLABu jako funkci generující zvukové soubory, poté zajistit jeho fungování v reálném čase díky audio toolboxu. Následně vytvořit VST plug-in a šablonu pro implementaci jednoduchých syntezátorů a návod k jejímu použití v jazyce C++ pomocí JUCE frameworku.

Práce se zabývá různými druhy syntéz a běžnými prvky digitálního syntezátoru. Základem syntéz je oscilátor generující periodický signál, který je různě upravován.

Teoretická část práce se zabývá generováním periodických průběhů a jednotlivými druhy syntéz. Dále jsou vysvětleny principy základních prvků digitálních syntezátorů, jako například kmitočtové filtry a generátor šumu.

V praktické části je ukázána implementace navrhovaného syntezátoru v prostředí MATLAB pracující v reálném čase, která zahrnuje i původní verzi syntezátoru pojatou jako funkce generující zvukové soubory. Dále se zaměřuje na syntezátor vytvořený pomocí C++ s využitím JUCE frameworku a popisuje šablonu pro implementaci jednoduchých syntezátorů.

1 Teoretická část studentské práce

Tato část se zabývá vybranými způsoby generování periodických průběhů. Dále je vysvětlen princip několika typů lineárních a nelineárních syntéz. Práce se také zabývá běžnými stavebními prvky syntezátorů, jmenovitě filtry, nízkokmitočtovými oscilátory, obálkou a generátorem šumu.

1.1 Generování periodických průběhů

Generovat periodické signály je možno například digitálním provedením analogových algoritmů, či způsoby používanými čistě digitálně.

Mezi analogové realizace patří harmonická syntéza, čítač modulo a integrace či derivace signálů [1].

Při digitálním generování periodických signálů se mezi možné řešení řadí rekurzivní oscilátory a přímá digitální syntéza, která pracuje na principu akumulace fáze s pevně danou či proměnnou vzorkovací frekvencí a fázovo-amplitudovým mapováním [3]. Je možné získat každý vzorek z matematické funkce vlny, avšak tento způsob je velmi neefektivní co se týče náročnosti na procesor a dobu výpočtu. Mnohem častěji se proto ukládají již vypočítané vzorky s předem určeným rozestupem do tabulky, kde je možné mezi nimi interpolovat [4].

V této kapitole je podrobněji vysvětlen čítač modulo a generování pomocí integrace a diferenciace původního signálu. Dále je zde zmíněn princip fungování rekurzivních oscilátorů a jejich nejzákladnější formy a přímá digitální syntéza.

1.1.1 Čítač modulo

Jedním z možných způsobů generování periodických signálů je modulo dělení. Nejprve se získá pilový průběh modulo dělením lineárně rostoucího časového vektoru požadovanou periodou, poté se výsledek vynásobí frekvencí, aby se zajistila nezávislost amplitudy na periodě a v posledním kroku se signál upraví tak, aby byl v rozmezí $\langle -1, 1 \rangle$. Toho se docílí jeho zdvojnásobením a odečtením 1, jak lze vidět v rovnici [1]:

$$x(n)_{saw} = 2(f \frac{n}{f_{vz}} \bmod 1) - 1. \quad (1.1)$$

Úpravou pilového průběhu se následně získají ostatní základní průběhy [1]

$$x(t)_{rect} = \text{sgn}[x(n)_{saw} - (2r_{DC} - 1)], \quad (1.2)$$

$$x(n)_{tri} = 2|x(n)_{saw}| - 1. \quad (1.3)$$

- f = požadovaný kmitočet signálu
- f_{vz} = vzorkovací kmitočet

1.1.2 Integrátor a diferenciátor

Počátečním signálem je obdélník či trojúhelník, následně se provede integrace či derivace a tímto způsobem se získá nový průběh [1].

Původní průběh	Integrace	Derivace
Obdélník	Trojúhelník	Bipolární sled impulsů
Trojúhelník	Parabola	Obdélník

Tab. 1.1: Generování signálů integrátorem a diferenciátorem

1.1.3 Rekurzivní oscilátory

Rekurzivní oscilátory jsou výhodné z hlediska výpočtu, avšak jsou schopny generovat pouze sinusový průběh a mohou mít nestabilní amplitudu či fázi a způsobovat růst kvantizačního šumu. [3]. Jsou založeny na simulaci reálného oscilátoru s nulovým tlumením a musí tedy dodržovat Barkhausenova kritéria, tedy celkový zisk cyklu musí být jedna a fázový posun celočíselný násobek 2π radiánů, upravena pro diskrétní čas [2].

Výpočet jednotlivých vzorků v diskrétním čase je založen na vzorci pro součet dvou goniometrických funkcí. Umožňuje získání následujícího vzorku ze dvou předchozích známých vzorků a úhlu reprezentující fázový skok mezi jednotlivými vzorky [2]. Po úpravě a zápisu rovnice v diskrétním čase získáme [3]

$$y(n+1) = 2\cos(\alpha)y(n) - y(n-1). \quad (1.4)$$

- $y(n)$ = momentální vzorek
- α = vzdálenost dvou sousedních vzorků
- $2\cos(\alpha)$ = fázový skok

Dvě základní formy rekurzivních oscilátorů jsou bikvadratická, neboli základní a párová [2]. Základní forma vychází z trigonometrické součtové rovnice pro sinus, je výpočetně méně náročná, avšak při změně frekvence se objeví změna amplitudy. To je možné vyřešit normalizací signálu, což způsobí nenávaznost fáze. Při použití základní formy je tedy nutno přepočítat počáteční podmínky pro každý nový vzorek [3]. Forma párová vychází ze součtové rovnice pro sinus a kosinus. Má problémy s nestabilitou amplitudy, ale není zde nutno přepočítávat počáteční podmínky.

1.1.4 Přímá digitální syntéza

Přímá digitální syntéza se používá pro generování vzorků složitějších tvarů vln. Základními parametry jsou fáze, která reprezentuje pozici vzorku v čase či v tabulce a amplituda. Fáze požadovaného vzorku se získá přičtením fázového inkrementu k původní hodnotě.[4]

Nejčastější provedení vypočítá hodnoty vzorků funkce požadované v předem určených intervalech. Přírůstek je buďto celočíselný, v tom případě se získá umístění předem vypočítaného vzorku, či neceločíselný. V případě neceločíselného fázového inkrementu je potřeba aproximovat hodnotu v závislosti na sousedních vzorcích. Celočíselný fázový přírůstek lze obecně vyjádřit [4]:

$$p = f \frac{N}{f_{vz}}. \quad (1.5)$$

- p = fázový přírůstek
- N = délka tabulky

Při známém fázovém posunu a amplitudě vzorku je možno vypočítat hodnotu vzorku vynásobením právě této amplitudy a pozice v tabulce určené novou hodnotou fáze.

1.2 Aditivní syntéza

Aditivní syntéza funguje na principu sčítání výstupů z více generátorů, výsledným zvukem je tedy jejich kombinace. Nejčastějším průběhem je sinus, zejména pokud se jedná o harmonickou syntézu, ale lze použít libovolný tvar vlny. Signál je po sečtení možno vyjádřit [5]:

$$x(t) = \sum_{n=1}^N \sum_{k=-\infty}^{\infty} c_{kn} e^{jk\omega_n t}. \quad (1.6)$$

- $x(t)$ = časový průběh signálu
- N = počet generátorů signálu
- c_{kn} = spektrální koeficient n -tého generátoru
- ω_{kn} = frekvence n -té harmonické
- ω_n = frekvence 1. harmonické

jak již bylo zmíněno, aditivní syntéza může být harmonická či neharmonická.

Harmonická syntéza spočívá ve sčítání sinusových signálů, jejichž frekvence jsou celočíselné násobky prvotního signálu. Amplitudy jednotlivých složek se mohou ponechat na stejné hodnotě, častěji se však postupně zmenšuje amplituda, aby se simulovalo chování harmonického spektra složitějších signálů. Harmonickou syntézou je možno vytvářet složité tvary vln, nejedná se však o nejefektivnější způsob.

Neharmonická aditivní syntéza nemá žádné podmínky na frekvence či amplitudy jednotlivých složek. Je možno kombinovat vlny se sinusovým průběhem a vlny se složitějším tvarem o různých frekvencích či přidat šum. Výsledkem je bohatější a zajímavější spektrum než u harmonické aditivní syntézy.

1.3 Subtraktivní syntéza

Subtraktivní syntéza využívá parametrických filtrů pro úpravu složitějšího signálu. Používat jako původní signál sinus nemá význam, jelikož filtrováním nelze získat zajímavější spektrum. U jednoduchých subtraktivních syntezátorů se využívá nejčastěji obdélník, trojúhelník či pila, díky svému bohatému spektru. Další možností je využít jako vstupní signál šum.

1.3.1 Filtrace při subtraktivní syntéze

Nejčastěji používané jsou IIR filtry typu horní a dolní propust, dále také pásmová propust, pásmová zadrž a hřebenový filtr [1]. Těmito filtry se zbavíme části spektra dané vlny, což změní výslednou barvu zvuku.

1.4 Frekvenční modulace

Základním principem FM je modulace frekvence nosného signálu signálem modulačním. Prozatím budeme předpokládat, že nosný i modulační signál je sinus. Poté je možno popsat FM rovnicí:

$$x(t) = A \sin(\omega_c t + I \sin \omega_m t), \quad (1.7)$$

kde

- $x(t)$ = okamžitá amplituda modulované vlny
- A = špičková amplituda modulované vlny
- ω_c = úhlová frekvence nosného signálu
- I = modulační index
- ω_m = úhlová frekvence modulačního signálu

1.4.1 Modulační index a deviační poměr

Modulační index, v procentech jako hloubka modulace, lze vyjádřit jako

$$D/f_m, \quad (1.8)$$

kde D je frekvenční zdvih, což je absolutní maximum rozdílu nosného signálu a signálu modulovaného. Je to také největší možná amplituda modulovaného signálu. Frekvence modulačního signálu, zde f_m , určuje rychlost modulační frekvence nosného signálu [6]. Při modulaci nosné vlny s nenulovým modulačním indexem vznikají postranní složky symetricky kolem f_c . Tyto postranní spektrální složky jsou celočíselné násobky modulační frekvence. S rostoucím indexem se zvětšuje šířka pásma, tudíž přibývá postranních frekvencí, a zároveň klesá amplituda nosné. Je tedy možné, že nebude nosná frekvenční složka ve spektru výsledné vlny přítomna.

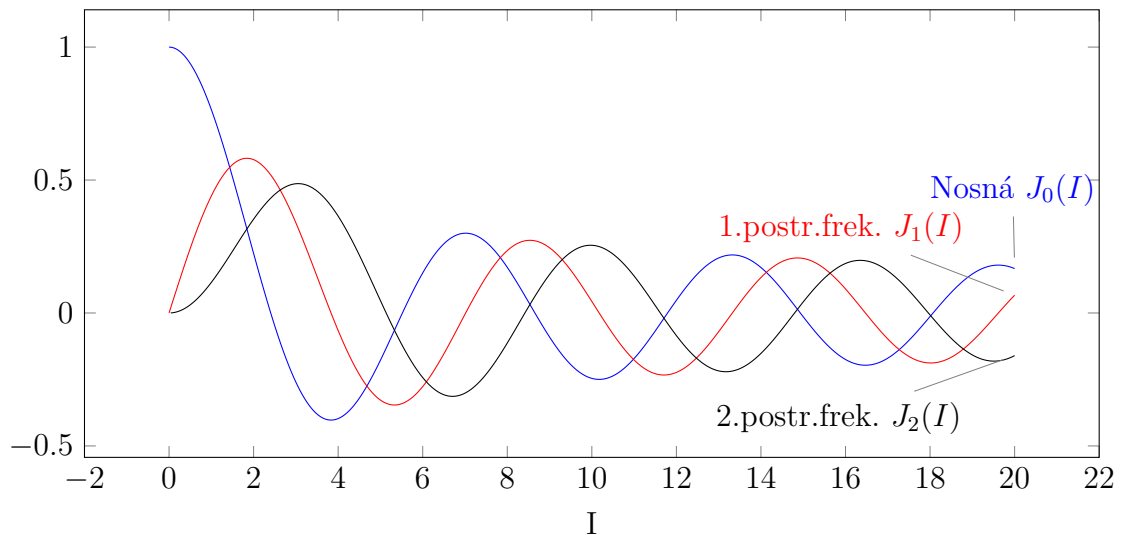
Deviační poměr $\frac{f_c}{f_m}$ určuje, jaké složky se objeví ve spektru. Aby výsledná vlna byla harmonická, musí být poměr racionální číslo. Při poměru 1:1 budou přítomny všechny složky, při 2:1 všechny sudé, 3:1 každá 3. etc. Neharmonické spektrum vzniká při iracionálním poměru [6].

1.4.2 Frekvenční pásma

Celková šířka pásma se dá přibližně určit následujícím vzorcem:

$$B \approx 2(D + f_m). \quad (1.9)$$

Pro zjištění amplitud nosné a frekvencí postranních pásem se používají Besselovy funkce prvního druhu, n -tého řádu $J_n(I)$ [6]. tyto amplitudy budou závislé na modulačním indexu.



Obr. 1.1: Besselovy funkce 1. druhu n -tého řádu

Fundament modulované vlny zjistíme z deviačního poměru:

$$\frac{f_c}{f_m} = \frac{N_1}{N_2}. \quad (1.10)$$

N_2 Pokud je spektrum harmonické, lze vypočítat pořadí jednotlivých harmonických složek tímto způsobem [6]:

$$k = |N_1 \pm nN_2|. \quad (1.11)$$

- k = harmonické číslo
- n = řád postranní frekvence, $n \in \mathbb{N}$

1.4.3 Lineární FM

Při lineární frekvenční modulaci se frekvence modulovaného signálu zvyšuje a snižuje o stejnou frekvenční výchylku, danou modulačním signálem. Postranní pásma budou symetrická podle nosné frekvence. [7]

1.4.4 Exponenciální FM

U exponenciální frekvenční modulace se využívají frekvenční skoky jako u not a oktáv. Při zvýšení o oktávu se tedy frekvence zdvojnásobí. Postranní pásma jsou asymetrická vůči nosné frekvenci. [7]

1.4.5 Fázová modulace

Při PM se moduluje fáze nosné vlny modulačním signálem. Při změně fáze vlny dojde i k variaci její frekvence a dá se říct, že se jedná o typ lineární FM [7]. Modulovaná PM vlna lze vyjádřit

$$x(t) = A \sin(\omega_c t + I \sin \omega_m t). \quad (1.12)$$

Frekvence je derivací fáze [7], pokud tedy chceme převést FM na PM, je nutno modulační signál před modulací integrovat

$$x(t) = A \sin(\omega_c t + I \int_0^t \sin \omega_m t \, dx). \quad (1.13)$$

1.5 Amplitudová modulace

Při amplitudové modulaci se amplituda nosného signálu mění podle okamžité amplitudy modulačního signálu [11]. Z modulačního signálu vznikne obálka pro signál nosný, podle které se řídí jeho amplituda. Vzorec pro výpočet okamžité amplitudy modulované vlny je

$$x(t) = A_c(1 + I \sin(\omega_m t)) \sin(\omega_c t), \quad (1.14)$$

kde

- $x(t)$ = okamžitá amplituda modulované vlny
- A_c = amplituda nosné vlny

- A_m = amplituda modulační vlny
- ω_c = úhlová frekvence nosného signálu
- I = modulační index
- ω_m = úhlová frekvence modulačního signálu

Ve výsledném signálu je tedy přítomen původní nosný signál, jehož amplituda je nezávislá na postranních pásmech [4].

1.5.1 Modulační index

Modulační index určuje výši modulace nosného signálu modulačním. Lze vyjádřit [8]:

$$I = \frac{A_m}{A_c}, \quad (1.15)$$

nebo také:

$$I = \frac{A_{\max} - A_{\min}}{A_{\max} + A_{\min}}, \quad (1.16)$$

kde:

$$A_{\max} = A_c + A_m \quad (1.17)$$

$$A_{\min} = A_c - A_m. \quad (1.18)$$

- $A_{\max}$ = maximální amplituda modulované vlny
- $A_{\min}$ = minimální amplituda modulované vlny

1.5.2 Frekvenční pásma

Při modulaci signálu vznikají postranní spektrální složky, jejich počet závisí na tvaru modulační vlny a poměru amplitud obou signálů.

Pokud je modulační index racionální, pak bude spektrum vzniklé amplitudovou modulací harmonické a vzniklé postranní složky budou násobkem frekvence nosné. Když se vyjádří modulační poměr jako zlomek dvou prvočísel, pak v případě že jmenovatel bude roven 1, budou přítomny všechny postranní složky. Při jmenovateli rovnému 2 budou přítomny pouze liché. Při iracionálním poměru jsou výsledná pásma neharmonická.[4]

Co se tvaru vlny týče, vhodnější jsou komplexnější tvary vln s bohatším spektrem, sinusový průběh vygeneruje jen 2 postranní složky.

Horní pásmo lze vypočítat následujícím způsobem:

$$A_c I \frac{1}{2} \sin((\omega_c + \omega_m)t), \quad (1.19)$$

dolní pásmo je dáno:

$$A_c I \frac{1}{2} \sin((\omega_c - \omega_m)t). \quad (1.20)$$

Výsledná modulovaná vlna je tedy kombinací obou pásem a nosné vlny [5]:

$$x(t) = A_c(1 + I \sin(\omega_m t)) \sin(\omega_c t) - A_c I \frac{1}{2} \sin((\omega_c + \omega_m)t) + A_c I \frac{1}{2} \sin((\omega_c - \omega_m)t). \quad (1.21)$$

Celková šířka pásma je:

$$B = 2f_m. \quad (1.22)$$

1.6 Kruhová modulace

Kruhová modulace je v podstatě amplitudová modulace s výjimkou chybějící frekvenční složky nosné ve výsledném signálu. Pokud počítáme, že oba signály mají sinusový průběh, tak bude po provedení modulace ve frekvenčním spektru nosného signálu původní složka nahrazena dvěma složkami o frekvenci $f_1 - f_2$ a $f_1 + f_2$ [4]. Výsledná funkce lze vypočítat pomocí rovnice:

$$x(t) = A_c(I \sin(\omega_m t)) \sin(\omega_c t), \quad (1.23)$$

1.7 Kmitočtové filtry

Kmitočtové filtry jsou lineární, časově invariantní spojité systémy [9]. Používají se k úpravě amplitudy spektrálních složek signálu v závislosti na jejich frekvenci. Filtry ovlivňují i časovou doménu, čehož se využívá například ve zpožďovacích efektech [11].

1.7.1 Parametrické filtry 1. řádu

Parametry využívané pro úpravu spektra jsou mezní (střední) kmitočet f_C , činitel jakosti Q a řád filtru [1]. Mezní kmitočet určuje jaká část kmitočtového pásma bude filtrem ovlivněna. Činitel jakosti ovlivňuje strmost průběhu útlumu na kmitočtu, čím je větší, tím strmější závislost je. Při moc velkém činiteli jakosti může dojít k rezonanci v oblasti mezního kmitočtu. Řád filtru určuje strmost a zvlnění filtru. Přenosová funkce filtru s kanonickou formou 1. řádu je [11]:

$$H(z) = \frac{b_0 + b_1 z^{-1}}{1 + a_1 z^{-1}}. \quad (1.24)$$

Typ filtru	b_0	b_1	a_1
Lowpass	$K/(K+1)$	$K/(K+1)$	$(K-1)/(K+1)$
Highpass	$1/(K+1)$	$-1/(K+1)$	$(K-1)/(K+1)$

Tab. 1.2: Koeficienty filtrů 1. řádu.

- $K_{\max} = \tan(\pi \frac{f_C}{f_{vz}})$
- f_{vz} = vzorkovací frekvence

Dolní propust

Zmenšuje amplitudu spektrálních složek pod mezním kmitočtem, kolem kterého mohou vzniknout rezonance [11], kvůli nárůstu přenosu kolem něj. Rezonanci je možné ovlivnit vhodnou jakostí filtru [1].

Horní propust

Ztlumuje složky na frekvencích pod f_C , opět je možné způsobit rezonanci kolem mezního kmitočtu [11].

1.7.2 Parametrické filtry 2. řádu

V první kanonické formě byly koeficienty b_2 a a_2 nulové [11]. Pokud mají nenulovou hodnotu, jedná se o 2. kanonickou formu, která se navíc k již zmíněným typům filtrů využívá pro pásmovou propust a pásmovou zadrž. Přenosová funkce je následující:

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}. \quad (1.25)$$

Typ filtru	b_0	b_1	b_2	a_1	a_2
Lowpass	$\frac{K^2 Q}{K^2 Q + K + Q}$	$\frac{2K^2 Q}{K^2 Q + K + Q}$	$\frac{K^2 Q}{K^2 Q + K + Q}$	$\frac{2Q(K^2 - 1)}{K^2 Q + K + Q}$	$\frac{K^2 Q - K + Q}{K^2 Q + K + Q}$
Highpass	$\frac{Q}{K^2 Q + K + Q}$	$-\frac{2Q}{K^2 Q + K + Q}$	$\frac{Q}{K^2 Q + K + Q}$	$\frac{2Q(K^2 - 1)}{K^2 Q + K + Q}$	$\frac{K^2 Q - K + Q}{K^2 Q + K + Q}$
Bandpass	$\frac{K}{K^2 Q + K + Q}$	0	$-\frac{K}{K^2 Q + K + Q}$	$\frac{2Q(K^2 - 1)}{K^2 Q + K + Q}$	$\frac{K^2 Q - K + Q}{K^2 Q + K + Q}$
Bandreject	$\frac{Q(1 + K^2)}{K^2 Q + K + Q}$	$\frac{2Q(K^2 - 1)}{K^2 Q + K + Q}$	$\frac{Q(1 + K^2)}{K^2 Q + K + Q}$	$\frac{2Q(K^2 - 1)}{K^2 Q + K + Q}$	$\frac{K^2 Q - K + Q}{K^2 Q + K + Q}$

Tab. 1.3: Koeficienty filtrů 2. řádu.

Pásmová propust

Propouští frekvence mezi 2 mezními kmitočty. Spektrální složky pod nižším a nad vyšším f_C jsou tlumeny [11].

Pásmová zádrž

Spektrální složky mezi nižším a vyšším mezním kmitočtem jsou potlačeny. Ostatní spektrální složky nejsou filtrem ovlivněny.

1.7.3 Hřebenový filtr

Tento typ filtru má periodicky se opakující odezvy. V případě IIR filtru prochází vstupní signál přes zpožďovací linku, která se vrací na vstup. Po každém cyklu se posune o konstantu g . Jeho přenosová funkce je

$$H(z) = \frac{c}{(1 - gz^{-M})}. \quad (1.26)$$

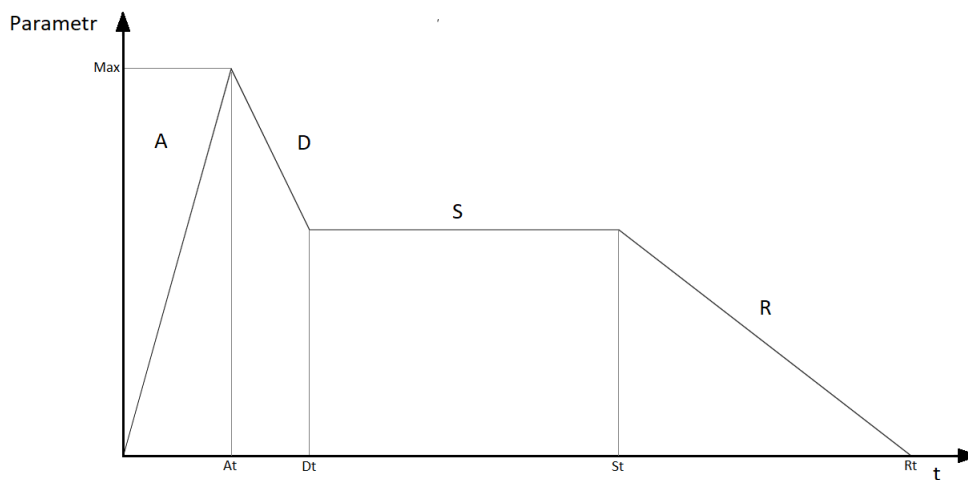
Na konci každého cyklu má zpožděný signál amplitudu g^p , kde p je momentální počet cyklů. Pro zajištění stability filtru se musí dodržet podmínka $|g| \leq 1$, díky které neroste signál do nekonečna [11].

1.8 LFO

Nízkofrekvenční oscilátor se pohybuje od desetin do desítek Hz a využívá se pro modulaci různých parametrů syntezátoru [10]. Příkladem je efekt tremola při modulaci frekvence, či vibráta při amplitudové modulaci. U filtrů ovládaných LFO se posouvá střední kmitočet, který sleduje amplitudu LFO. U hřebenového filtru se využívá pro efekt phaser. Další využití je například změna umístění v prostoru či ovládání rychlosti arpeggia. Nejběžnější jsou základní typy vln, ale často se používají i složitější tvary. LFO může být i větší počet.

1.9 ADSR obálka

ADSR obálka je dalším způsobem, jakým je možno modulovat signál. Vybraný parametr se řídí podle obálky, která se skládá ze 4 nastavitelných částí.



Obr. 1.2: ADSR obálka

Attack

Parametr Attack vyjadřuje čas, za který se dostane parametr na své maximum.

Decay

Vyjadřuje čas, za který parametr poklesne z maxima na ustálenou hodnotu

Sustain

Doba po kterou se parametr drží na ustálené hodnotě

Release

Release time je doba, za kterou daný parametr klesne z ustálené hodnoty na nulu.

Dalšími parametry obálky jsou Initial level a End level, které nastavují počáteční a konečnou úroveň. ADSR obálka může být navrhnutá samostatně a poté použita pro změnu parametrů, či být vytvořena generátorem obálky signálem Gate z klaviatury [12], čehož se využívá pro získání přirozenějšího průběhu zvuku. V případě ovládání z klaviatury je Sustain doba, po kterou je držena klávesa.

Existují i další typy obálek, například AHDSR, která má navíc parametr Hold, který vyjadřuje dobu, po kterou se hodnota drží na maximu. AHDBDR, kde se nachází 2 fáze Decay po sobě a AHDBDSR, kde po 2. Decay fázi klesne úroveň na hodnotu Sustain a pokračuje na Release [12].

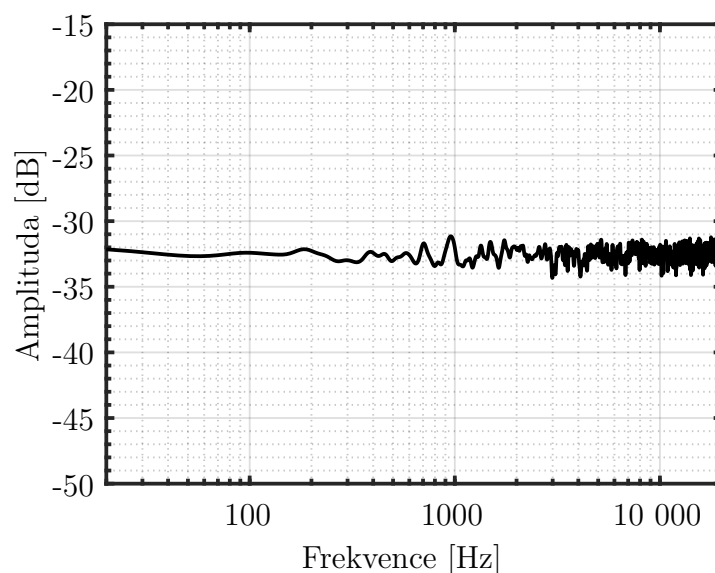
Modulaci obálkou je možno implementovat lineárně či exponenciálně. Častější provedení je exponenciální, jelikož zní lidskému uchu přirozeněji.

1.10 Generátor šumu

Generátor šumu vytváří bílý šum, který se poté buď přimíchává k výslednému signálu, nebo filtruje. Bílý šum se vytváří například pomocí generátorů náhodných hodnot [10]. Filtrací bílého šumu vzniká barevný šum, který dělíme podle poklesu či zvyšování výkonu v závislosti na frekvenci.

1.10.1 Bílý šum

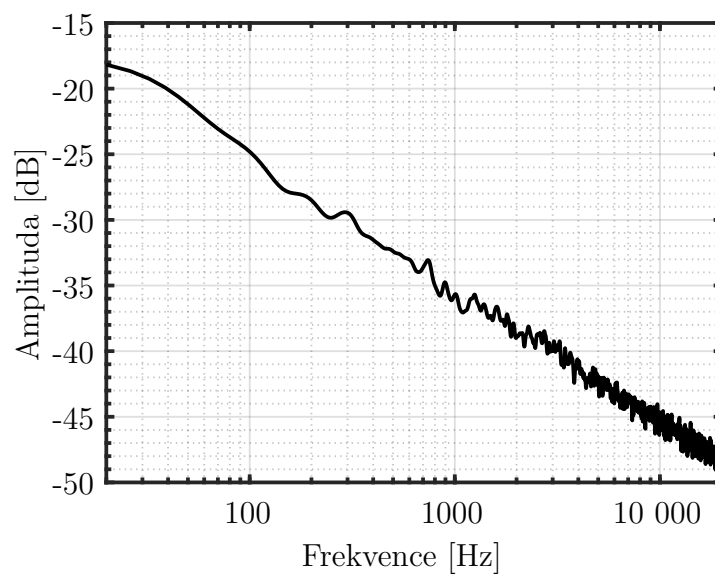
Má konstantní výkon po celém spektru, z čehož vyplývá že se skládá ze všech možných harmonických složek, které jsou přibližně rovnoměrně rozloženy.[10]



Obr. 1.3: Výkonové spektrum bílého šumu

1.10.2 Růžový šum

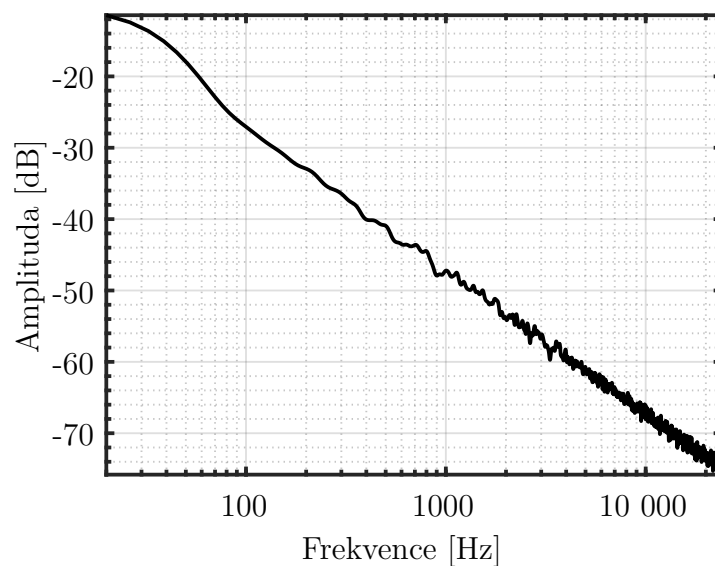
Klesá o 3 dB na oktávu a jeho výkonová hustota je úměrná obrácené frekvenci $1/f$ [14].



Obr. 1.4: Výkonové spektrum růžového šumu

1.10.3 Červený šum

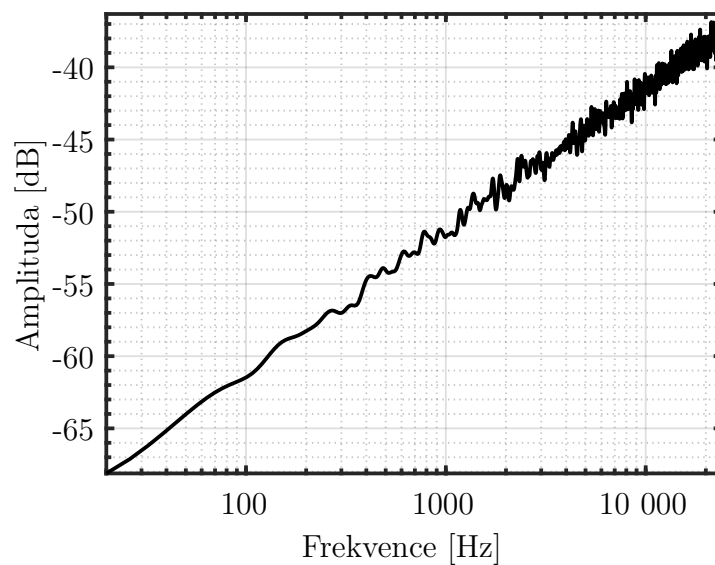
Také zvaný hnědý šum, klesá o 6 dB na oktávu. Jeho výkonová hustota je $1/f^2$ [14].



Obr. 1.5: Výkonové spektrum červeného šumu

1.10.4 Azurový šum

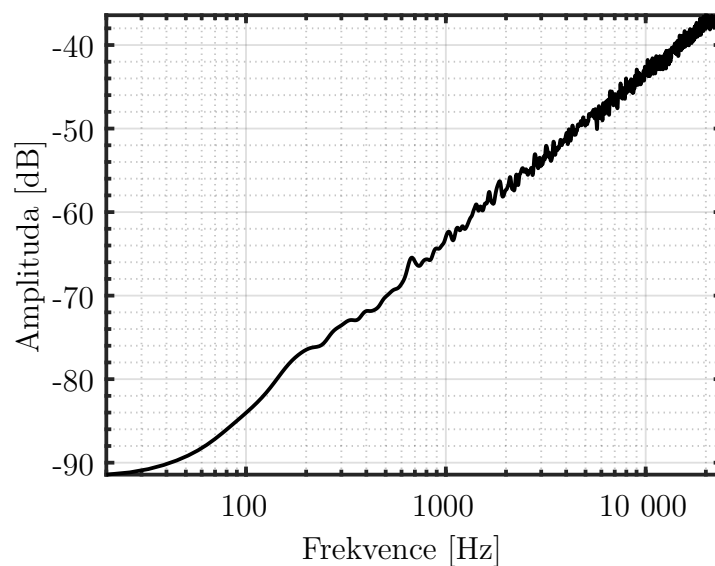
Jeho spektrální hustota stoupá 3 dB na oktávu, je tedy rovna frekvenci [14].



Obr. 1.6: Výkonové spektrum azurového šumu

1.10.5 Purpurový šum

Výkonová hustota purpurového šumu stoupá o 6 dB na oktávu a je tedy f^2 [14].



Obr. 1.7: Výkonové spektrum purpurového šumu

2 Implementace syntezátoru v prostředí MATLAB

Tato kapitola se zaměřuje na testování syntezátoru v prostředí MATLAB. K jeho vytvoření byla použita verze R2023a. Jedná se o implementaci jednotlivých funkcí a vztahů mezi nimi, které byly nejprve testovány samostatně a poté jako součást syntezátoru pracujícího v reálném čase. Pro návrh byly použity rozšíření audio toolbox, signal processing toolbox a DSP system toolbox. V této implementaci byly otestovány 4 druhy syntézy, LFO, jednoduchá obálka, parametrické filtry a generátor šumu.

2.1 Generování zvukových souborů

Testování generování základních tvarů vln, filtrů a modulací vln bylo nejprve provedeno v prostředí MATLAB jako funkce vytvářející zvukové soubory podle zadaných hodnot. Tato funkce funguje na principu oken obsahujících možnosti pro výběr a nastavení hodnot parametrů, které určí výsledný tvar vlny. Výstup určené délky se normalizuje a uloží do souboru pojmenovaného uživatelem.

2.1.1 Jednoduchý oscilátor

Byly testovány dva způsoby generování základních průběhů, první využíval MATLAB funkce generující přímo vzorky dané vlny o periodě 2π . Vstupním parametrem byla časová množina vzorků, pro které měla být vlna vygenerována. Druhým způsobem byl výpočet pomocí čítače modulo. Pro implementaci pracující v reálném čase byl vybrán druhý způsob generování vln.

2.1.2 Filtry

Pro implementaci filtrů byl nejprve vygenerován daný typ filtru s arbitrárními parametry a následně byla funkce upravená tak, aby bylo možné zadávat vlastní vstupní proměnné a vytvářet filtry dle potřeb uživatele. Takto upraveny byly funkce pro dolní, pásmovou a horní propust.

Dále byla testována možnost přidání neharmonických složek pomocí filtrace bílého šumu hřebenovým filtrem vyššího řádu. Takto upravený šum byl přidán k výstupu oscilátoru. Přičítání neharmonických složek nebylo ve verzi pracující v reálném čase a C++ implementaci provedeno z důvodu složitosti výpočtu potřebného hřebenového filtru.

2.1.3 Implementace syntéz

Aditivní syntéza

Aditivní syntéza byla ve verzi nepracující v reálném čase testována sčítáním daného počtu vln se sinusovým průběhem. Při výběru tohoto typu syntézy v testovací funkci se vybere počet vln k sečtení a následně proběhne smyčka pro každý jednotlivý oscilátor. V každé smyčce se vygeneruje sinusový průběh dané frekvence, který je uložen do 1D matice obsahující pole vzorků jednotlivých vln. Tímto způsobem se matice zaplní a poté se její složky postupně sečtou, čímž vznikne výsledný signál.

Subtraktivní syntéza

Při výběru subtraktivní syntézy se zadají parametry oscilátoru, jehož vzorky se následně vygenerují. Poté je nutno zvolit počet filtrů a jejich typ a kmitočty odpovídající filtrovanému pásmu. Výstup oscilátoru se filtruje všemi filtry zapojenými do kaskády.

Amplitudová modulace

Po zvolení možnosti amplitudové modulace je možno vybrat průběhy nosného i modulačního oscilátoru a modulační index. K výstupu nosného oscilátoru se přičte jeho součin s výstupem oscilátoru modulačního a modulačního indexu.

Frekvenční modulace

Pro frekvenční modulaci je možno zvolit jako nosnou pouze vlnu se sinusovým průběhem, modulační oscilátor je schopen všech základních tvarů vlny. Časový vektor je proveden lineárním vektorem jdoucím od 0 do hodnoty součinu vzorkovacího kmitočtu a požadované délky zvukového souboru. Modulovaná frekvence je vypočtena přičtením hodnot vzorků modulační vlny vynásobené modulačním indexem k nosnému kmitočtu.

$$x(t) = A_c \sin((2\pi f_c t) + I x_{\text{mod}}) \quad (2.1)$$

2.1.4 Generátor šumu

Byly testovány dva způsoby generování barevného šumu. Vždy se vychází z bílého šumu, který je poté buď filtrován, nebo je přímo upraveno jeho amplitudové spektrum tak, aby se získal barevný šum s frekvenčním spektrem odpovídajícím požadovaným podmínkám [14]. Pro bílý šum je použita funkce vracující náhodné hodnoty, které se následně normalizují.

Růžový šum je generován přímou úpravou spektra bílého šumu. Do proměnné se uloží náhodné hodnoty a provede se FFT, následně se zjistí počet unikátních bodů ve spektru a vytvoří se vektor indexů. Odmocninou tohoto vektoru se vydělí amplitudy pouze levé části spektra, jelikož pravá část je její komplexně sdružená kopie [14]. Provede se rekonstrukce spektra. Podobným způsobem je řešen červený a purpurový šum.

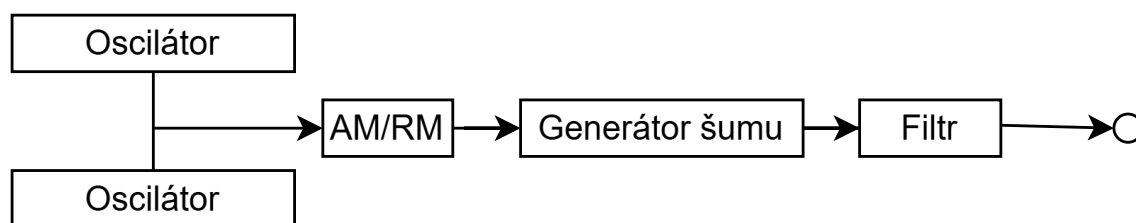
Azurový šum opět využívá generování náhodných hodnot. tyto hodnoty jsou poté filtrovány tak, aby jejich spektrální hustota stoupala se 3 dB na oktávu. Pro tento účel je použita inverzní dolní propust, která by filtrací vyústila v růžový šum.

Z popsaných metod generování šumu byla pro verzi pracující v reálném čase zvolena filtrace bílého šumu z důvodu jednoduchosti implementace.

2.1.5 LFO

V rámci testování generování zvukových souborů byl implementován nízkofrekvenční oscilátor pro modulaci amplitudy. Amplitudové spektrum vlny se vynásobí se vzorky nízkofrekvenčního oscilátoru a poté proběhne normalizace a vytvoření souboru.

2.2 Implementace pracující v reálném čase



Obr. 2.1: Blokové schéma návrhu Matlab syntezátoru

2.2.1 Jednoduchý oscilátor

Průběhy generované oscilátorem se získávají pomocí modulu čítače popsaného v podkapitole 1.1.1. Je tedy možné vybrat pilový, trojúhelníkový nebo obdélníkový tvar. Sinusový průběh je generován pomocí MATLAB funkce `sin`.

Vstupními parametry jsou požadovaný tvar vlny, její frekvence, vzorkovací kmitočet, velikost bloku vzorků, počáteční hodnota bloku vzorků a střída.

Časový vektor se získá ze vzorkovací frekvence a velikosti bloku vzorků.

2.2.2 Implementace filtrů

Je implementován filtr se stavovými proměnnými 2. řádu. Koeficienty se počítají pro zvolený typ filtru dle tabulky 1.3 před začátkem zpracovávání vstupu. Filtr samotný využívá MATLAB funkci *filter*, které se předají koeficienty, vstup určený k filtraci a počáteční podmínky filtru. Tyto podmínky jsou při prvním cyklu nulové a poté se každý následující cyklus přepisují výstupními podmínkami z předešlého cyklu. Filtrace signálu poté probíhá těsně před výstupem.

Střední kmitočet filtru je možno modulovat pomocí modulačního oscilátoru. Hodnota oscilátoru společně s původním středním kmitočtem filtru nastaveným uživatelem se používá při výpočtu koeficientů.

2.2.3 Implementace syntéz

Pro MATLAB verzi syntezátoru pracující v reálném čase byla otestována amplitudová a kruhová modulace společně s aditivní a subtraktivní syntézou. Zdroj nosného signálu je vždy první oscilátor, modulační zdroj je poté oscilátor druhý.

Aditivní syntéza

Verze pracující v reálném čase má 2 oscilátory, jejichž výstupy jsou sečteny dle rovnice 1.6. K součtu dochází až po provedení modulace 1. oscilátoru.

Subtraktivní syntéza

Sečtený výstup obou oscilátorů se filtruje filtrem se stavovými proměnnými 2. řádu dle zadaných parametrů.

Amplitudová modulace

Vzorky obou oscilátorů se vzájemně vynásobí a ponechá se nosná, vychází se z rovnice 1.14.

2.2.4 Generátor šumu

Základem pro všechny typy šumu je bílý šum, vygenerovaný jako množina rovnoměrně rozložených náhodných čísel. Tento šum se následně filtruje tak, aby jeho amplitudové spektrum klesalo či stoupalo dle hodnot uvedených v popisu každého šumu v podkapitole 1.10. Růžový a červený šum vznikají filtrací dolní propustí. Purpurový šum je filtrován horní propustí a azurový šum je filtrován horní propustí vzniklou invertováním dolní propusti pro růžový šum.

2.2.5 Uživatelské rozhraní

Uživatelské rozhraní je zprostředkováno pomocí MATLAB objektu Plugin interface, který slouží pro napojení elementů uživatelského rozhraní na hodnoty parametrů.

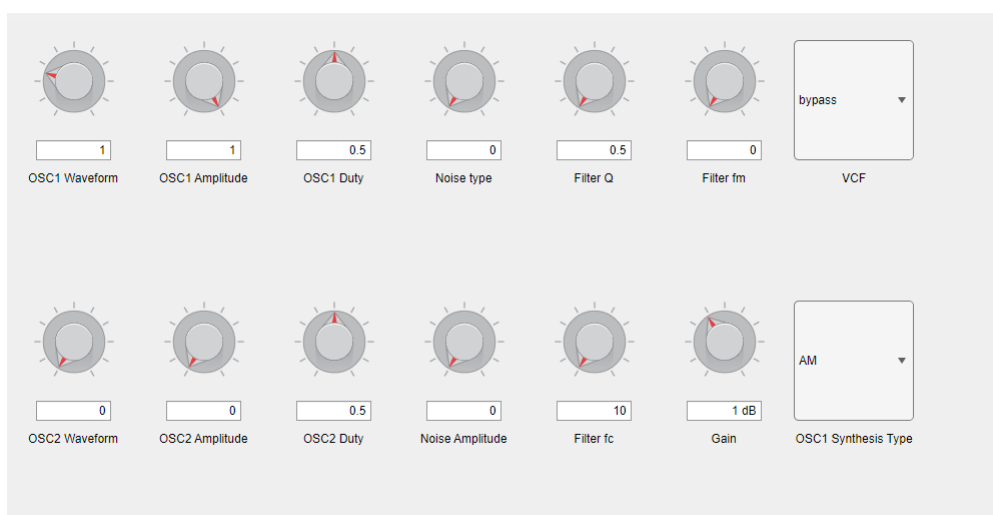
K dispozici jsou dva oscilátory s nastavitelným průběhem, střídou a amplitudou signálu. Frekvence je určena MIDI notou přijatou ze zdroje MIDI, který se nastavuje přímo v kódu do proměnné *MIDI_device*, kterou je nutno přepsat na momentálně připojený ovladač.

Šumový generátor má ovladatelnou výstupní úroveň a je možno vybrat bílý, růžový, azurový nebo purpurový šum.

Filtr má nastavitelný mezní kmitočet, který lze modulovat pomocí parametru *Filter fm*, jakost a typ. Vybrat lze z dolní propusti, pásmové propusti, horní propusti a pásmové zadrže.

Parametr *OSC1 Synthesis Type* obsahuje možnosti modulace prvního oscilátoru druhým oscilátorem pomocí amplitudové nebo kruhové modulace.

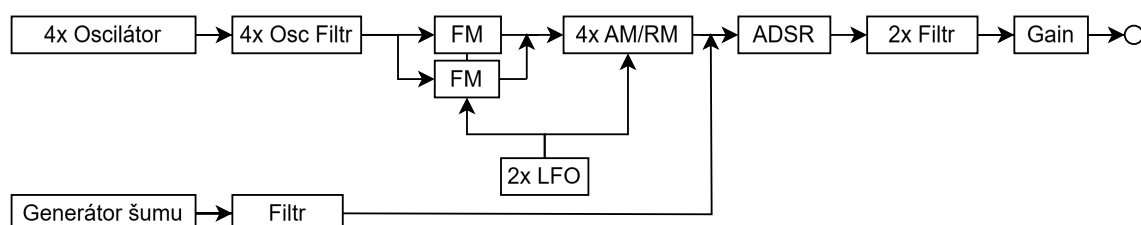
Úroveň výsledného signálu je nastavena parametrem *Gain*.



Obr. 2.2: MATLAB uživatelské rozhraní syntezátoru

3 Implementace syntezátoru v jazyce C++

Tato kapitola zahrnuje praktickou část práce zabývající se implementací testovaných algoritmů z předchozí kapitoly pomocí jazyka C++ a JUCE frameworku. Obsahuje stručný popis tohoto jazyka a rozhraní, návrh a provedení syntezátoru jako plug-in nástroje pracujícího v reálném čase a vysvětluje šablonu pro implementaci jednoduchých syntezátoru v JUCE.



Obr. 3.1: Blokové schéma návrhu VST syntezátoru

3.1 Jazyk C++

C++ je objektivě orientovaný jazyk, čehož bylo při implementaci využito k tvorbě přehledné a snadno rozšiřovatelné struktury syntezátoru. K psaní kódu bylo použito Visual Studio 2022.

3.2 JUCE framework

Struktura syntezátoru vychází z JUCE frameworku. Z knihovny s ním dostupné se využívají funkce pro zpracování audio bloků a vzorků v reálném čase, filtry, ADSR, gain, základ pro třídu oscilátorů a prvky uživatelského rozhraní. Framework zprostředkovává kompilaci a vytvoření VST3 plug-in modulu a samostatného VST3 nástroje.

3.2.1 Projucer

Generování projektu a nastavení typu plug-inu, název a zpracování MIDI vstupu a výstupu pro použití v DAW bylo provedeno v Projuceru.

3.3 Struktura

Struktura syntezátoru byla pojata s myšlenkou jednotlivých bloků, které se dají zapínat, vypínat a upravovat dle potřeby.

Kód je rozdělen do 3 hlavních složek. První složka **Data** obsahuje třídy určené pro generování, úpravu signálu, zpracovávání dat a funkcí pro nastavení parametrů. Druhá část kódu je obsažena ve složce **Processing**, která se stará o zpracování signálu a správné přiřazení hodnot parametrů uživatelem. **UI** složka obsahuje všechny komponenty tvořící grafické rozhraní nástroje a editor, který zprostředkovává jejich korektní zobrazení.

3.3.1 Generování signálu

Pro generování zvukového signálu jsou v syntezátoru implementovány 4 oscilátory a šumový generátor. Zvuk je vytvářen samostatně pro každý momentálně aktivní hlas syntezátoru.

Oscilátor

Syntezátor má vlastní třídu dědící z JUCE třídy pro oscilátory.

Způsob generování vzorků oscilátorů vychází z kombinace wavetable oscilátoru a přímé digitální syntézy popsané v kapitole 1.1.4. JUCE zprostředkovává akumulaci fázových přírůstků v závislosti na vzorkovacím kmitočtu a kmitočtu požadovaném. Oscilátor je inicializován matematickou funkcí, která se namapuje v rozmezí $\langle -\pi, \pi \rangle$. Nachází se zde možnost vytvoření tabulky hodnot s nastavitelným počtem vzorků. Fázový přírůstek je vypočten následovně

$$p = 2\pi \frac{f}{f_{vz}}. \quad (3.1)$$

Pro získání pozice nadcházejícího vzorku se k momentální fázi přičte přírůstek a odečte π . Následně se vypočte hodnota vzorku či se načte z pozice tabulky určené fází.

Pro sinusový průběh je použita jeho matematická funkce. Pila je vygenerována pomocí lineárně rostoucí funkce, která se periodicky překlopí na 0 a poté začne opět růst. Trojúhelník je realizován tabulkou obsahující 128 vzorků. Vzorky jsou přemapované z $\langle -\pi, 0 \rangle$ na $\langle -1, 1 \rangle$ a z rozmezí $\langle 0, \pi \rangle$ na $\langle 1, -1 \rangle$. Střídáním kladných a záporných vzorků stejné hodnoty vznikne obdélníkový průběh se střídou 50%.

Změna frekvence se provádí detekcí noty stisknuté klávesy, ke které se přičte hodnota rozladění. Třída oscilátoru dále obsahuje funkce pro nastavení parametrů filtru a výstupní úrovně.

Šumový generátor

Šumový generátor je obsažen ve třídě `NoiseGeneratorData`. Bílý šum se generuje po vzorcích z náhodných hodnot v omezeném rozmezí. Růžový šum je generován pomocí aproximace filtrování bílého šumu dolní propustí s přesností 0,5 dB na oktávu. Úroveň výsledného vzorku se nastaví na požadovanou hodnotu a poté jde na výstup. Funkce `prepareToPlay` je nutná kvůli správnému nastavení vzorkovací frekvence, počtu vzorků v bloku a výstupních kanálu pro `gain`.

Aliasing

Generování signálu základními oscilátory není pásmově omezeno a z toho důvodu dochází k aliasingu, který je nejvíce znatelný u obdélníkového a pilového průběhu. Tento problém by bylo možné vyřešit generováním pásmově omezeného signálu, nebo pomocí nadvzorkování, filtrace anti-aliasingovou dolní propustí a podvzorkováním.

V rámci syntezátoru byla testována metoda převzorkování, která však způsobila značné zpomalení výpočtů vyúsťující ve velmi znatelné artefakty na výstupu. Generování pásmově omezeného signálu nebylo testováno, s největší pravděpodobností by se však jednalo o vhodnější způsob zamezení aliasingu.

3.3.2 Modifikátory signálu

Mimo filtry obsažené v blocích generátoru signálu, je pro jeho úpravu použita ADSR obálka a 2 filtry zapojené v sérii. Syntezátor umožňuje modulaci hlavních oscilátorů pomocí 2 nízkofrekvenčních oscilátorů.

Obálka

ADSR obálka se vytváří a aplikuje pomocí JUCE třídy `ADSR`, ze které dědí vlastní třída syntezátoru `Envelope`. Obsahuje pouze konstruktor, který přijímá jako vstupní proměnné hodnoty parametrů `attack`, `decay`, `sustain` a `release` vyobrazené na obr. 1.2.

Obálka přechází z jednoho stavu ADSR do dalšího dle hodnot nastavených uživatelem. Fáze obálky se ovládají pomocí proměnné získané vydělením maximální amplitudy dané fáze násobkem vzorkovacího kmitočtu a časem v sekundách nastaveným uživatelem. Attack fáze se provádí postupným zvyšováním amplitudy vzorků až do maximální hodnoty, poté se přejde do fáze decay. Hodnoty vzorků obálky klesají podle času nastaveném uživatelem parametrem `DT` od maxima až k dosáhnutí `sustain` hodnoty. Amplitudy Vzorků se až do konce této fáze nemění. Poslední částí je `release`, která funguje na stejném principu jako `decay` s konečnou hodnotou v 0.

Po výpočtu obálky se její blok vynásobí s blokem dat z výstupu oscilátorů a šumového generátoru. K jejímu spuštění dojde při stlačení klávesy, po uvolnění se provede fáze release.

Filtr

Třída objektů zprostředkující filtry se nazývá `FilterData` a pracuje na bázi JUCE objektu třídy `StateVariableTPFilter`. Jedná se o 12 dB/oct IIR filtr využívající TPT (topology preserving structure transform) strukturu popsanou Vadimem Zavalishin [13]. Tato struktura byla navržena na bázi analogového filtru se stavovými proměnnými, její největší výhodou je potlačení slyšitelných artefaktů při změně hodnoty mezního kmitočtu. Filtry jsou díky této vlastnosti vhodné pro rychlé změny a modulaci středního kmitočtu. Při případné modulaci středního kmitočtu by však stále bylo potřebné modulační signál vyhladit, jelikož nelze plně předejít slyšitelnému praskání při velkých, náhlých skocích ve středním kmitočtu.

Je přítomna funkce pro výběr typu filtru podle vstupní proměnné, příprava filtru, funkce pro nastavení parametrů filtru a pomocné funkce pro zpracování vstupu.

Po zavolání funkce pro zpracování vstupního vzorku se vygenerují pomocí uživatelem nastavených hodnot koeficienty filtru pro všechny typy filtru a vrátí se vzorek filtrovaný typem zvoleným.

Nízkofrekvenční oscilátor

Třída nízkofrekvenčních oscilátorů dědí z vlastní třídy syntezátoru `OscManager`. Obsahuje funkce pro nastavení průběhu a frekvence do 50 Hz. Výstup je v případě pilového průběhu vyhlazen, aby nedošlo k příliš velkým skokům hodnot způsobujících slyšitelné praskání při modulaci. Generovaný signál je použit pouze pro modulaci hlavních oscilátorů a nejde na výstup.

3.3.3 Syntézy

Syntezátor umožňuje aditivní, subtraktivní syntézu a amplitudovou, kruhovou a frekvenční modulaci. Syntézy probíhají pro každý hlas zvlášť a jsou tedy obsaženy v kódu pro zpracování bloku zvukových dat hlasu.

Aditivní syntéza

Aditivní syntéza se provádí po jednotlivých vzorcích. Vzorky vygenerované všemi oscilátory a šumovým generátorem se po provedení veškerých modulací sečtou a předají se výstupnímu bloku.

Subtraktivní syntéza

Každý z hlavních oscilátorů se filtruje IIR filtrem 1. řádu. Šumový generátor má filtr 2. řádu. Další filtrace probíhá těsně před výstupem 2 sériově zapojenými IIR filtry 2. řádu. Všechny filtry kromě filtrů hlavních oscilátorů mají TPT strukturu.

Frekvenční modulace

Jako první po vygenerování vzorků a filtrace oscilátorů probíhá frekvenční modulace. Syntezátor obsahuje 2 FM bloky zprostředkující nastavení a hlídající aktivitu.

Nejprve se zjistí, jestli je alespoň 1 z bloků aktivní. Pokud ano, tak se daný blok uloží do řady a přiřadí se k němu hodnota hloubky modulace a zdroj nosného a modulačního signálu. Vzorky nosného signálu jsou poté přepsány. Tato operace se provádí pro každý aktivní FM objekt zvlášť, pokud nemají nastavený stejný zdroj nosného signálu. V případě že se zdroj shoduje, amplitudy modulačních signálů vynásobené jim odpovídajícími hloubkami modulace se sečtou a hodnota je použita jako modulační vzorek. Poté se nastaví nová frekvence a oscilátorem se vygeneruje vzorek, který se předá k amplitudové modulaci, či jde na výstup. Pokud se zdroje nosného signálu liší, modulace se provedou paralelně nezávisle na sobě.

Amplitudová modulace

Amplitudová modulace je provedena sériově až 4 AM/RM bloky, dochází k ní až po dokončení frekvenční modulace. Po normalizaci amplitudy modulačního vzorku na hodnotu v rozmezí $\langle 0, 1 \rangle$ se vzorek nosného signálu vynásobí se vzorek modulačního signálu a hodnotou hloubky modulace, nakonec je přičten nemodulovaný vzorek nosného signálu.

Výběr libovolné kombinace zdroje nosného a modulačního signálu je docíleno pomocí 2 polí obsahujících vzorky zpracované jednotlivými oscilátory a nízkofrekvenčním oscilátorem. Každý modulátor a generátor nosného signálu má svůj index, podle kterého se vybere požadovaný vzorek pro zpracování.

Kruhová modulace

Kruhová modulace se provádí stejným způsobem jako amplitudová, ale neobsahuje neupravený vzorek nosného signálu. Obě modulace se mohou měnit a kombinovat v libovolném pořadí.

3.3.4 Zpracovávání signálu v reálném čase

PluginProcessor se stará o zpracování audio dat. Po spuštění programu se nastaví vzorkovací frekvence, počet kanálu, velikost bufferu dle zařízení a inicializují

se všechny hlasy. Blok zvukových dat je rozdělen a postupně se zpracovává. Nejprve se nastaví všechny parametry, poté se zpracuje blok funkcí `RenderNextBlock`, ve které probíhá zpracování jednotlivých vzorků bloku pro všechny hlasy syntezátoru. Po tom co syntezátor dostane MIDI zprávu `NoteOn`, se nastaví frekvence oscilátoru, ke které je přičtena hodnota rozladění a proběhne filtrace. Provedou se modulace vzorků, sečtou se výstupy oscilátorů a šumového generátoru a aplikuje se ADSR obálka. Výsledný blok je poté postupně upraven oběma filtry.

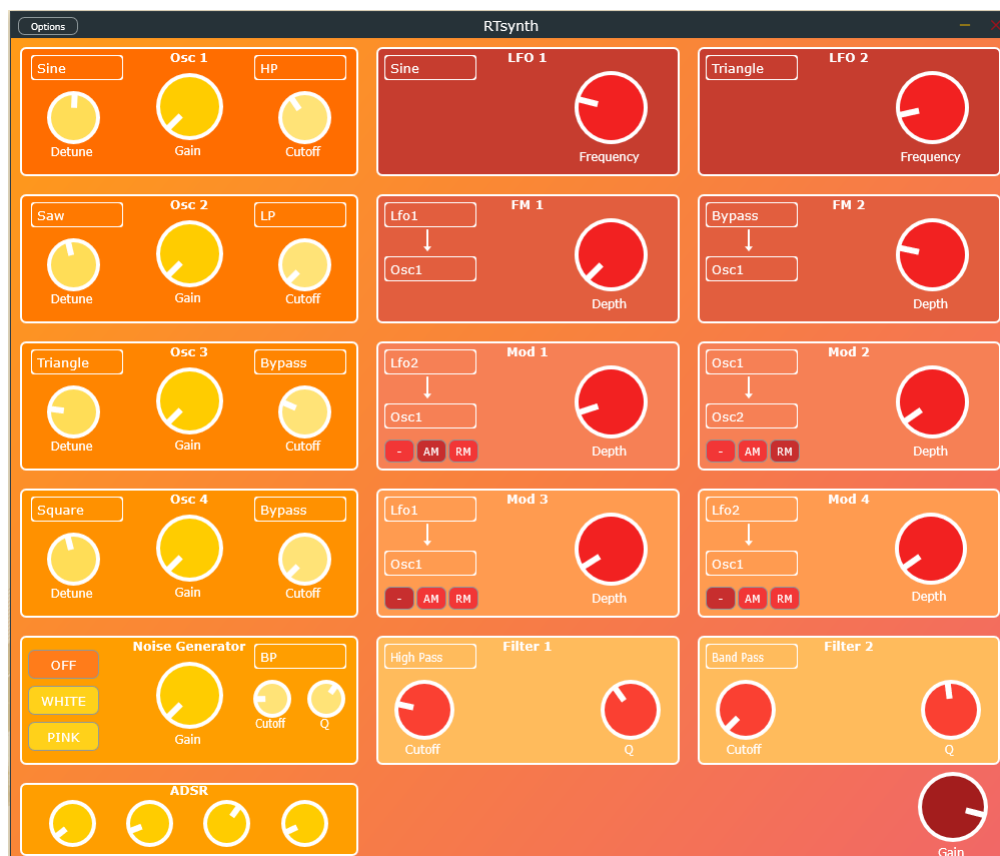
3.4 Uživatelské rozhraní VST plug-inu

Uživatelské rozhraní syntezátoru bylo pojato z hlediska přehlednosti a znázornění cesty signálu od vstupu k výstupu. K jeho realizaci byla použita třída `Component`, ze které se vycházelo při tvorbě vlastních tříd prvků uživatelského rozhraní.

V levé části se nachází část dedikovaná generování signálu. Pravá část je částí modulační. Výstup oscilátorů je modulován od vrchu dolů, nejprve se provede frekvenční modulace a poté amplitudová a kruhová. Jako poslední se signál filtruje a poté jde na výstup, jehož úroveň se ovládá otočným potenciometrem.

Syntezátor obsahuje 4 laditelné oscilátory s nastavitelnou úrovní a IIR filtrem 1. řádu. Každý oscilátor má možnost volby tvaru vlny mezi sinem, pilou, trojúhelníkem a obdélníkem se střídou 50%. Ve spodní části se nachází šumový generátor schopný bílého a růžového šumu. Má, obdobně jako oscilátory, nastavitelnou úroveň a IIR filtr 2. řádu s laditelným středním kmitočtem a činitelem jakosti. Poslední blok této sekce obsahuje ADSR obálku s potenciometry umožňujícími nastavení jednotlivých částí nezávisle na sobě.

V modulační části se nachází nízkofrekvenční oscilátory schopné všech základních průběhů a nastavitelnou frekvencí do 50 Hz. Frekvenční modulace se ovládá pomocí dvou bloků, pracujících paralelně. jako nosný oscilátor je možné zvolit jakýkoli základní oscilátor, modulátor může být každý z těchto oscilátorů a navíc oscilátory nízkofrekvenční. Blok dále umožňuje změnu hloubky modulace. Modulační bloky slouží k sériové amplitudové či kruhové modulaci, které se vybírají spodními tlačítky. Volba zdroje nosného a modulačního signálu a hloubky modulace funguje stejně jako u frekvenčního modulátoru. Před výstupem jsou 2 IIR filtry 2. řádu s nastavitelným středním kmitočtem a činitelem jakosti.



Obr. 3.2: Uživatelské rozhraní VST syntezátoru

3.4.1 Vykrešlování komponentů uživatelského rozhraní

K finálnímu vzhledu potenciometrů se došlo přepsáním JUCE třídy `LookAndFeel_V4`, která zprostředkovává vykreslování uživatelského rozhraní na obrazovku. Vzhled rozbalovacího menu je proveden funkcí `paint` nacházející se ve třídě `CustomComboBox`. Tlačítka jsou realizována pomocí úpravy třídy `TextButton`.

Vlastní třída `LookAndFeel` obsahuje pouze funkci pro vykreslení otočného potenciometru a nastavení jeho barvy. Funkci se předávají rozměry potenciometru a reference na jeho objekt. Jako první se vykreslí obvod kružnice bílou barvou a poté se vyplní podle nastavené barvy. Barvu potenciometru je nutno zvolit před samotným počátkem vykreslování objektu. Možné hodnoty parametru potenciometru se přemapují na kružnici. Vytvoří se úzký obdélník, který se následně otočí tak, aby ukazoval na potenciometru na momentální hodnotu parametru, která se ukáže uprostřed kružnice v případě že se na ní nachází kurzor. Nakonec se pod parametr vypíše jeho název.

Ve třídě pro rozbalovací menu je potřeba pouze přepsat funkci `paint` tak, aby barvy odpovídali zbytku uživatelského rozhraní. Tyto barvy je opět potřeba nastavit

před začátkem vykreslování komponentu.

Tlačítka použitá v syntezátoru jsou upravená tak, aby bylo možné mít aktivní pouze jedno ze skupiny sdružených tlačítek. Je možné nastavit zvlášť barvu zapnutého a vypnutého tlačítka.

3.4.2 Napojení komponentů uživatelského rozhraní

Každý ovladatelný prvek má vlastní komponent. Pro dosažení uniformního vzhledu prvků a usnadnění přidávání nových elementů se využívají třídy `CustomComponent` a `CustomSlider`. Konečný vzhled je nastaven v `PluginEditor`, kde se inicializují všechny komponenty a přiřadí se k nim parametry, které ovládají. Definuje se zde také celková velikost okna syntezátoru a velikost a vzájemné umístění bloků komponentů.

3.5 Šablona pro implementaci jednoduchých syntezátorů v JUCE

Součástí práce je šablona obsahující soubory pro implementaci jednoduchého syntezátoru a návod k jejímu použití. V rámci šablony je projekt již vytvořen, nicméně v případě použití šablony pouze jako vodítko k jinému projektu obsahuje i základní informace o frameworku JUCE a Projuceru, popisuje jejich instalaci a použití při práci v editoru VisualStudio a vysvětluje jak vytvořit a nastavit nový projekt.

V základním stavu má syntezátor šablony oscilátor generující sinusový průběh a jednoduché uživatelské rozhraní. Další soubory jsou nachystány pro vytvoření vlastní třídy nastavující ADSR obálku, filtr, výstupní úroveň a jejich prvky uživatelského rozhraní.

První část šablony se zaměřuje na vysvětlení struktury syntezátoru a jeho základních tříd. Nejprve je popsána třída zprostředkující právě zpracování zvukových dat v reálném čase, která obsahuje objekt starající se o ukládání parametrů a všechny objekty aktivních hlasů. Následně je vysvětleno fungování hlasu syntezátoru, jak hlas nastavit a připravit ke zpracování generovaného signálu.

V podkapitole o komponentech se pojednává o třech vlastních třídách základních součástí syntezátoru, tedy oscilátoru, filtru a obálky. Je zde uvedeno, ze které JUCE třídy jsou odvozeny a vysvětlují se jejich metody a proměnné.

Část zabývající se uživatelským rozhraní odůvodňuje použití vlastních tříd pro každý komponent uživatelského rozhraní a vysvětluje propojení mezi rozhraním a reálnými hodnotami parametrů syntezátoru. Obsahuje obecný popis třídy komponentu, jejich objekty a metody potřebné ke správnému fungování. Nakonec je zmí-

něna třída starající se o inicializaci a správnému zobrazení uživatelského rozhraní `PluginEditor` a je zde přiložen příklad vzhledu hotového jednoduchého syntezátoru.

Třetí kapitola je podrobný návod ke zprovoznění polyfonního syntezátoru s jedním oscilátorem schopným několika průběhů vlny, ADSR obálkou spuštěnou stlačením klávesy, IIR filtrem 2. řádu s TPT strukturou a potenciometrem pro ovládání výstupní úrovně. Každý komponent má svou podkapitolu, kde jsou vysvětleny potřebné kroky ke správnému generování či zpracování signálu. Popisují se potřebné kroky k dokončení třídy komponentu a pro jeho připojení k signálovému řetězci. Také se zde nachází způsob vytvoření prvku uživatelského rozhraní a jeho napojení k reálnému parametru. Některé podkapitoly obsahují návrhy pro vylepšení či rozšíření syntezátoru, které nejsou v návodu popsány a jsou myšleny jako samostatné menší projekty pro uživatele pracujícího dle šablony.

Šablona byla vytvořena pro uživatele, kteří nemají žádné předchozí zkušenosti s frameworkem JUCE, předpokládá se základní schopnost programování v jazyce C++. Je tedy vhodná jak pro výuku, tak jako osobní projekt pro uživatele, kteří chtějí začít ve frameworku JUCE pracovat.

Závěr

Práce se zabírala návrhem a implementací syntezátoru v prostředí MATLAB jako funkce generující zvukové soubory, jako modul pracující v reálném čase s využitím audio toolboxu a dále jako vst plug-in pomocí jazyku C++ a JUCE frameworku. Další součástí práce byla šablona pro implementaci jednoduchých syntezátorů pomocí JUCE frameworku, ke které byl vytvořen návod popisující práci s tímto frameworkem a způsob vytvoření základního syntezátoru.

Obě implementace syntezátoru pracující v reálném čase a syntezátor vytvořen pomocí šablony mají k dispozici grafické uživatelské rozhraní pro ovládání parametrů všech prvků.

Výstupem práce je VST plug-in syntezátor, který podporuje několik typů syntéz, jmenovitě aditivní, subtraktivní, amplitudové modulace, kruhové modulace a frekvenční modulace, disponující několika filtry se stavovými proměnnými, nízko-frekvenčními oscilátory, šumovým generátorem a ADSR obálkou.

Šablona vytvořená v rámci práce popisuje způsob implementace syntezátoru obsahujícího oscilátor se základními průběhy vln, ADSR obálku, filtr se stavovými proměnnými a potenciometr pro ovládání výstupní úrovně.

Literatura

- [1] SCHIMMEL, Jiří. *Hudební elektronika: Digitální realizace analogových syntéz zvuku*, Online. Brno: Vysoké učení technické v Brně [cit. 2023-5-7]
- [2] TURNER, Clay S. Recursive Discrete-Time Sinusoidal Oscillators. *IEEE Signal processing magazine*. 2003, vol. 20, no. 3, s. 103
- [3] SYMONS, Pete. *Digital Waveform Generation*. Cambridge University Press, 2014. ISBN 978-1-107-020979.
- [4] DE POLI, Giovanni. A Tutorial on Digital Sound Synthesis Techniques. *Computer Music Journal*. 1983, vol. 7, no. 4, s. 8-26.
- [5] SCHIMMEL, Jiří. *Hudební elektronika: Analogové syntézy zvuku*. Online. Brno: Vysoké učení technické v Brně [cit. 2023-8-7]
- [6] CHOWNING, John M. Synthesis of Complex Audio Spectra by Means of Frequency Modulation. *Journal of the audio engineering society*. 1973, vol. 21, no. 7, s. 526-534. New York, Audio engineering society, 1973.
- [7] A SIF, Shaik. Amplitude Modulation. Online. *Physics and radio electronics*. Dostupné z: <https://www.physics-and-radio-electronics.com/blog/amplitude-modulation/>. [cit. 2023-12-07].
- [8] NIELSEN, Kasper. PRACTICAL LINEAR AND EXPONENTIAL FREQUENCY MODULATION FOR DIGITAL MUSIC SYNTHESIS. Online. International Conference on Digital Audio Effect. 2020, s. 132-137. Dostupné z: https://dafx2020.mdw.ac.at/proceedings/papers/DAFx2020_paper_61.pdf. [cit. 2024-02-03].
- [9] SMÉKAL, Zdeněk. *Analýza systémů a soustav: Systémy se spojitým časem*. Online. Brno: Vysoké učení technické v Brně [cit. 2024-02-15]
- [10] PIRKLE, Will C.. *DESIGNING SOFTWARE SYNTHESIZER PLUGINS IN C++ WITH AUDIO DSP*. 2. Routledge, 2021. ISBN 978-0-367-51046-6.
- [11] ZÖLZER, Udo (ed.). *DAFX: Digital Audio Effects*. 2. Hamburg: Wiley, 2011.
- [12] SCHIMMEL, Jiří. *Hudební elektronika: Analogové syntézátory*. Online. Brno: Vysoké učení technické v Brně [cit. 2024-02-16]
- [13] ZAVALISHIN, Vadim. *THE ART OF VA FILTER DESIGN*. Online. rev. 2.1.0. 2018. Dostupné z: <https://www.native-instruments>.

com/fileadmin/ni_media/downloads/pdf/VAFilterDesign_2.1.0.pdf?
srsltid=AfmB0opPdjhPUNa6VHS2EOBunC1UR-a1qhbF17jbVIOF4whRMXSaLLL-.
[cit. 2025-04-23].

- [14] ZHIVOMIROV, Hristo. A Method for Colored Noise Generation. *RJAV*. 2018, vol. 15, no. 1, s. 17-16. ISSN 1584-7284

Seznam symbolů a zkratek

AM	amplitudová modulace
B	šířka pásma
DP	dolní propust
f_C	střední kmitočet
f_c	nosný kmitočet
FM	frekvenční modulace
f_m	modulační kmitočet
f_{vz}	vzorkovací kmitočet
HP	horní propust
I	modulační index
IIR	infinite impulse response - nekonečná impulzní odezva
LFO	low frequency oscillator - nízkofrekvenční oscilátor
PP	pásmová propust
PM	fázová modulace
PZ	pásmová zadrž
Q	činitel jakosti
TPT	topology preserving structure transform
VST	Virtual Studio Technology

Seznam příloh

A	Kód MATLAB implementace syntezátoru	45
B	Zdrojový kód a VST3 plug-in syntezátoru	46
C	Zdrojový kód šablony s pdf souborem návodu	47

A Kód MATLAB implementace syntezátoru

V příloze je obsažena složka obsahující MATLAB verzi syntezátoru nepracující v reálném čase `MATLAB_synth`. Uvnitř se nachází soubory funkcí všech prvků syntezátoru a hlavní funkce `synth.m` určené k ovládání parametrů syntezátoru a generování zvukových souborů. Verze pracující v reálném čase je obsažena ve složce `MATLAB_RTsynth`. Nachází se v ní hlavní funkce syntezátoru společně s pomocnými funkcemi například oscilátoru nebo šumového generátoru.

B Zdrojový kód a VST3 plug-in syntezátoru

Zdrojový kód plug-in syntezátoru obsahuje kód všech tříd komponentů syntezátoru, tříd pro zpracování signálu a nastavení hodnot parametrů, tříd komponentů uživatelského rozhraní a editoru, který je používán k jejich zobrazení. V příloze je dále soubor VST3 samotného plug-in nástroje.

C Zdrojový kód šablony s pdf souborem návodu

Příloha obsahuje zdrojový kód, Projucer projekt a pdf soubor s návodem k šabloně.